

Red Team Testing for Recommendation Systems Driven by Large Language Models

Design and Evaluation of Rag-Poison-Lab

Sbaaoui Idriss
Student ID: 22060004

Computer Science, 华东理工大学
Supervisor: 张恒润

- Problem and motivation in LLM-driven recommendation pipelines.
- Harmful retrieval poisoning threat model.
- Rag-Poison-Lab workflow: baseline vs attacked comparison.
- Related work supporting recommender poisoning and RAG poisoning.
- Technical route: metrics, attacks, and experiment pipeline.
- Experimental environment: Docker, backend, Elasticsearch, Kibana, LLM re-ranking.
- Sustainability and budget plan.
- Thesis schedule.

- LLMs are now part of many recommendation pipelines for retrieval, re-ranking, and explanation generation.
- This proposal studies **red team testing for recommendation systems driven by large language models**.
- The thesis system is Rag-Poison-Lab, built to track poisoning signals from retrieval to final output.
- Evaluation uses HR@K, NDCG@K, MRR@K, ASR@K, candidate overlap, and target-rank lift.
- Main question: where can an actor attack earliest, and how much can final output be manipulated?

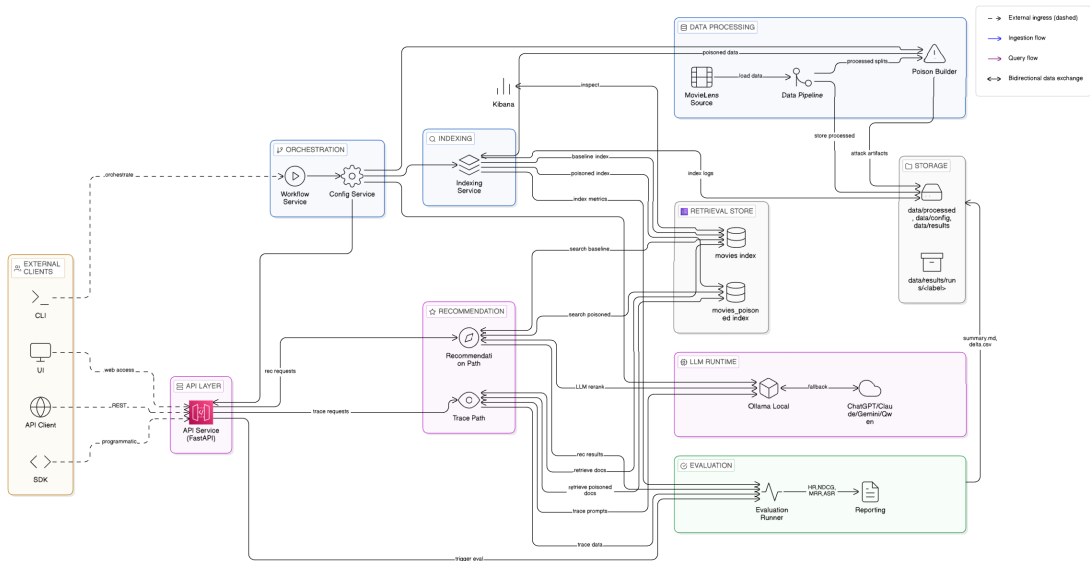
Research Background: RAG-Based Recommendation Systems

- The thesis is built around Rag-Poison-Lab with fixed user request and two retrieval modes.
- User profile is converted to a retrieval query, candidates are retrieved from Elasticsearch, and top- K items are ranked.
- If retrieval changes, final model behavior also changes in RAG pipelines.
- The system supports both deterministic ranking and LLM re-ranking for direct comparison.
- Dataset: MovieLens 100K, chosen for benchmark quality and local reproducibility.

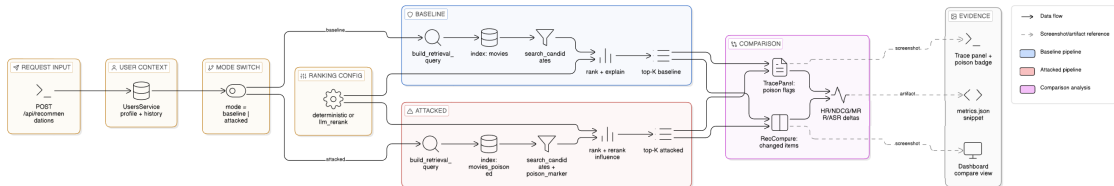
Threat Model: Harmful Retrieval Poisoning

- The attacker does not need to retrain the model.
- The attacker can poison indexed data to push intended retrieval candidates.
- This can shift ranked output and recommendation explanations.
- Planned attack types in the codebase:
 - Targeted promotion
 - Prompt injection payload insertion
 - Untargeted degradation

Rag-Poison-Lab Architecture



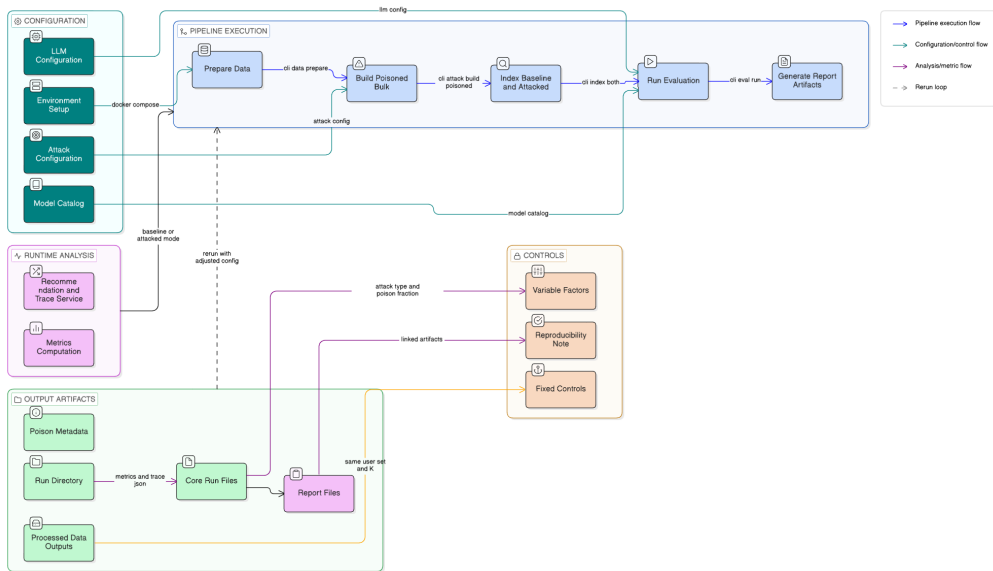
Baseline vs Attacked Workflow



- Recommender poisoning attacks: factorization, graph recommenders, top- N , and deep recommenders.
- RAG poisoning and security: poisoned retrieval knowledge can shift downstream behavior.
- Recommendation-specific RAG poisoning: poisoned retrieval context can manipulate final recommendations.
- Ranking evaluation theory: HR, MRR, and NDCG remain strong choices for ranked recommendation evaluation.

- Controlled comparison settings: same users, same K , same ranking mode, same processed data snapshot.
- Core metrics: $HR@K$, $NDCG@K$, $MRR@K$.
- Attack-sensitive metrics: $ASR@K$, candidate overlap at K , target-rank lift.
- Attack configuration lives in `attack_config.json`.
- Experiment loop:
 - 1 Prepare baseline processed data
 - 2 Generate poisoned bulk from baseline bulk
 - 3 Index baseline and attacked bulks separately
 - 4 Run recommendation requests
 - 5 Compute metrics and export artifacts

Experiment Pipeline Figure



Experimental Environment: Backend Stack

- Docker Compose services: Elasticsearch, Kibana, Ollama, RagPoison, and indexer.
- Health checks and startup ordering are used to reduce runtime failures.
- Backend API routers: health, users, recommendations, trace, and LLM settings.
- Fixed retrieval flow:
 - build query from user profile,
 - choose index by mode (`movies` vs `movies_poisoned`),
 - retrieve candidates,
 - rank with deterministic or `llm_rerank`,
 - return items and debug evidence.
- CLI workflow: data, attack, index, eval, report.

- Elasticsearch is the core retrieval layer for this thesis.
- Retrieval uses weighted `multi_match`: `title3`, `genres2`, `synopsis`, with seen-item filtering.
- BM25 similarity is used in both baseline and attacked index mappings.
- Kibana is used to inspect index health, document counts, and poisoning marker/payload fields.
- LLM runtime supports deterministic mode and `llm_rerank` mode.
- If re-ranking fails, fallback to deterministic ranking is recorded for traceability.

- Local-first execution keeps compute modest and avoids heavy always-on cloud workloads.
- Reused processed artifacts and hash-based freshness checks reduce unnecessary reruns.
- Staged runs (single/batch/full) reduce wasted compute during debugging.

Item	Cost (RMB)
Local electricity	400
Storage and backup	500
OpenAI API credit	800
Claude API credit	1000
Gemini API credit	800
Qwen API credit	400
Miscellaneous buffer	250
Total	4150

- **March 2026:** verify baseline reproducibility, and run pilot single-user tests.
- **April 2026:** run batch/full evaluations for all attacks under deterministic and LLM re-rank modes.
- **May 2026:** analyze metrics and traces, finalize figures/tables, complete writing.
- **Early June 2026:** supervisor revision cycle, reproducibility re-check, final submission.

Thank You

Questions and feedback are welcome.